

5-Day Functional Programming Foundations Course

Table of Contents (Facilitator)

- 1. [Facilitator Preparation](#)
- 2. [Day 1: Data, Variables & Operators - Facilitator Guide](#)
- 3. [Day 2: Expressions, Statements & Conditions - Facilitator Guide](#)
- 4. [Day 3: Loops, Lists & Data Structures - Facilitator Guide](#)
- 5. [Day 4: Functions, Scope & Errors - Facilitator Guide](#)
- 6. [Day 5: Input/Output, Algorithms & Complete Design - Facilitator Guide](#)
- 7. [Appendix: Answer Keys](#)
- 8. [Appendix: Props & Setup](#)

Facilitator Preparation

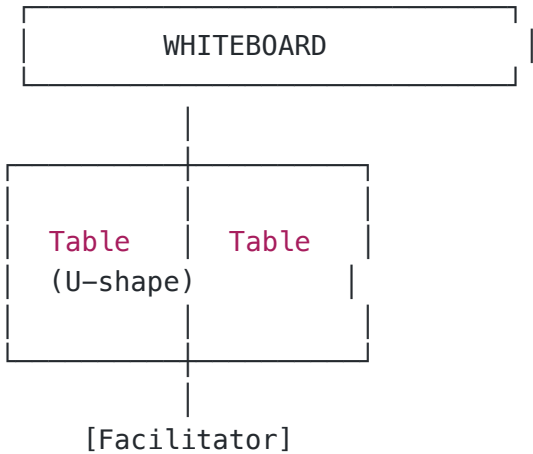
Master Props Checklist

Item	Quantity	Purpose	Day Used
Index cards (5 colors)	500 total	Variables, data types, functions	All
Sticky notes (various sizes)	10 pads	Temporary labeling	All
Whiteboard + markers	1 large	Diagrams and group work	All
Colored paper (5 colors)	100 sheets	Data type sorting	Day 1
Envelopes (A6)	50	"Box" simulation	Day 1
Play money	1 set	Cashier exercise	Day 2
Stopwatch/timer	1	Activity timekeeping	All
Blindfolds	5	Scope game	Day 4
Cardboard boxes (5)	5	Data type sorting bins	Day 1
String/yarn	20m	Pipeline visual connections	Day 4
Name tags (blank)	30	Role-play identification	All
Flip chart paper	10 sheets	Group designs	Day 5
Markers (assorted)	20	Drawing exercises	All

Item	Quantity	Purpose	Day Used
Dice	5	Randomization exercises	Day 3
Green/Red cards	30 each	Expression/Statement game	Day 2

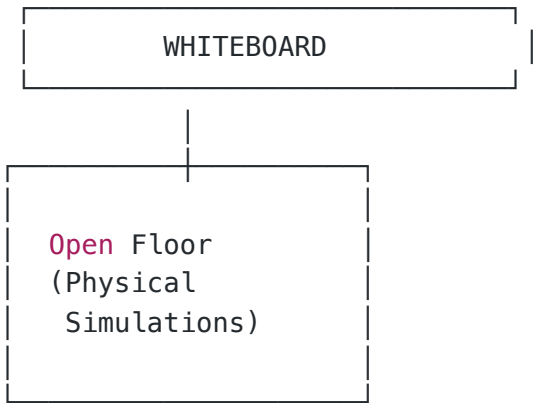
Classroom Setup Guide

Day 1-2: U-Shape Configuration



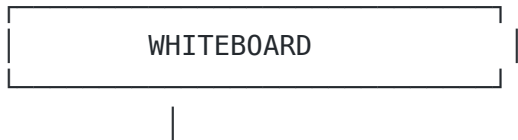
Benefits: Face-to-face interaction, easy eye contact, group work possible.

Day 3: Open Floor Space



Benefits: Allows movement for physical simulations, card passing, role-play.

Day 4-5: Group Tables



Grp1	Grp2	Grp3	Grp4

Benefits: Team collaboration, project work, presentations.

Day 1 Facilitator Guide

Data, Variables & Operators

Session Plan Overview

Time	Activity	Duration
09:00-09:15	Opening & Icebreaker	15 min
09:15-09:45	Data Types Theory	30 min
09:45-10:15	EXERCISE: Sort the Boxes	30 min
10:15-10:45	Variables Theory	30 min
10:45-11:15	EXERCISE: Variable Lockers	30 min
11:15-11:45	Operators Theory	30 min
11:45-12:15	EXERCISE: Calculator Game	30 min
12:15-13:00	LUNCH	45 min
13:00-14:00	EXERCISE: Spot the Type Error	60 min
14:00-15:00	BUILD: Complete Data Map	60 min
15:00-15:30	Day 1 Wrap-Up	30 min

Morning Session Script

09:00 - 09:15 | Opening & Icebreaker

[SAY] "Good morning everyone. Before we begin, let me ask you a question. **Raise your hand if you've ever given instructions to someone and they did it wrong.** "

(Wait for hands)

"Now raise your hand if you've ever followed a recipe and it turned out terrible."

(Wait for laughs)

"Today we're going to understand WHY that happens. Programs are just instructions. And like recipes, they fail when we mix up ingredients or use the wrong tools."

[WRITE ON WHITEBOARD]

Today's Goal: Understand WHAT we work with
and WHAT we can DO with it.

09:15 - 09:45 | Data Types Theory

[SAY] "Let's start with a fundamental question. If I gave you a toolbox, what would you find inside? "

(Collect answers: hammer, screwdriver, nails, screws)

"Exactly. Now here's the problem. Can you hammer a screw? "

(They say no)

"Can you screw a nail?"

(They say no)

"Programming is EXACTLY the same. Different tools for different jobs. In programming, we call these tools **DATA TYPES**."

[SETUP] Place 5 cardboard boxes on a table, each labeled:

Box 1	Box 2	Box 3	Box 4	Box 5
NUMBERS	TEXT	BOOLEANS	LISTS	NOTHING
				

[SAY] "Let me explain each box:"

Box 1 - NUMBERS *(Hold up a number card)* "Numbers are exactly what you think. 5, 3.14, -10, 1000. You can add, subtract, multiply, divide. Like money, age, temperature."

Box 2 - TEXT (String) *(Hold up a text card)* "Text is any letters, words, or sentences. 'Hello', 'John', 'The sky is blue'. Always in quotes. You can combine them, make them uppercase, count letters."

Box 3 - BOOLEANS (*Hold up true/false cards*) "True or false. That's it. Like a light switch. On or off. Yes or no. Is it raining? True or false. Are you over 18? True or false."

Box 4 - LISTS (Array) (*Hold up a list card*) "A list holds MULTIPLE things in order. Like a shopping list: ['milk', 'eggs', 'bread']. Or numbers: [1, 2, 3, 4, 5]. You can count them, get the first one, add more."

Box 5 - NOTHING (Null/Undefined) (*Hold up blank card*) "This is... nothing. Empty. Missing. Like an empty seat on a bus. It exists, but there's no value there. We'll use this rarely, but you need to know it exists."

[WRITE ON WHITEBOARD]

Data Types:

1. Number: 5, 3.14, -10
2. Text: "Hello", "John"
3. Boolean: true, false
4. List: [1,2,3], ["a","b"]
5. Nothing: null, undefined

09:45 - 10:15 | EXERCISE: Sort the Boxes

[SAY] "Now YOU will sort these items. Each of you gets a bag of mixed items. Your task: **Put every item in the correct box.** "

[DISTRIBUTE] Give each participant a bag containing:

- 5 number cards: 5 , 3.14 , -10 , 100 , 0
- 5 text cards: "Hello" , "John" , "Blue" , "123" , "True"
- 5 boolean cards: true , false , true , false , true
- 5 list cards: [1,2,3] , ["a","b"] , [] , [true,false] , [10,20,30]
- 3 nothing cards: null , undefined , null

[TIMER] Set timer: 10 minutes

[FACILITATOR WALKTHROUGH] Walk around and observe. Common mistakes:

- "123" as text vs 123 as number → **Explain:** Quotes make it text!
- [] as nothing → **Explain:** Empty list is still a LIST (just empty)
- "True" vs true → **Explain:** Quotes make it text!

[DEBRIEF - 5 minutes] Go through each box and confirm correct sorting. Have participants call out what they put where.

[SAY] "Excellent. Now, underneath each box, write down THREE operations you can do with that type."

Whiteboard Guide (Fill in with them):

Type	Operations
Numbers	Add, subtract, multiply, divide
Text	Combine (Hello + World), uppercase, count letters
Booleans	AND, OR, NOT
Lists	Count items, get first item, add items
Nothing	Check if empty (is it null?)

10:15 - 10:45 | Variables Theory

[SAY] "Now that we know WHAT things are, we need to NAME them. **Think about your school locker. You don't say 'the metal box on the third floor'. You say 'MY locker'.** That's a variable."

[WRITE ON WHITEBOARD]

Variable = A NAME that points to a VALUE

Example: `age = 25`
`name = "John"`
`isStudent = true`
`scores = [85, 90, 78]`

[SAY] "In programming, we give names to our data. So instead of writing '5' everywhere, we write 'age'. And if age changes... wait. **In functional programming, age NEVER changes.** "

[DRAMATIC PAUSE]

"Let me repeat that. **Never.** If you want to change age, you don't update the locker. You get a NEW locker with a NEW name."

[DEMONSTRATE]

[WRITE ON WHITEBOARD]

WRONG (mutation): `age = 25` → `age = 26` (erased!)

RIGHT (immutable): `age = 25` → `newAge = 26` (kept both!)

[SAY] "Why? Because if I gave you my locker and said 'age is 25', and then later said 'age is 26', you'd be confused. WHICH age? The old one? The new one? By keeping both, we have a PERFECT history."

10:45 - 11:15 | EXERCISE: Variable Lockers

[SETUP] Give each participant:

- 10 index cards
- 1 piece of paper (their "locker")
- 1 marker

[SAY] "Here's your assignment. I'll call out values. You write them on a card, give it a name, and put it on your 'locker' paper."

[CALL OUT - One at a time]

1. "Number 25, name it 'age'"
2. "Text 'John', name it 'firstName'"
3. "Boolean true, name it 'isActive'"
4. "List [85, 90, 78], name it 'scores'"

[SAY] "Now I want you to CHANGE age from 25 to 26. BUT... you cannot erase the old card. What do you do?"

(Participants should write a NEW card: `age2 = 26` or `newAge = 26`)

[SAY] "Good! Now you have both cards. This is called **IMMUTABILITY**."

[WRITE ON WHITEBOARD]

IMMUTABLE = Cannot be changed once created.
Always create NEW data instead.

[SAY] "Why is this important? Think about a bank. If I change my balance from \$100 to \$50 by ERASING, we lose the history. But if I say 'Transaction: Withdraw \$50, newBalance = \$50', we have a PERFECT RECORD. This is why banks use immutability."

11:15 - 11:45 | Operators Theory

[SAY] "We have data. We have names for data. **Now what can we DO with it?** This is where OPERATORS come in."

[WRITE ON WHITEBOARD - Large, Permanent]

Type	Operators	Example	Result
Arithmetic	+ - * / %	5 + 3	8
Comparison	> < == != >= <=	10 > 5	true
Logical	AND OR NOT	true AND false	false
Text	+ .length	"Hello" + "!"	"Hello!"
List	+ .length	[1,2] + [3,4]	[1,2,3,4]

[SAY - Pointing to each row]

- **Arithmetic:** Math operations. You know these.
- **Comparison:** Asking questions like "Is 10 bigger than 5?" → Answer: true or false.
- **Logical:** Combining true/false. "Is it raining AND do I have an umbrella?" → Only true if BOTH are true.
- **Text:** You can combine text or count letters.
- **List:** You can combine lists or count items.

[DEMONSTRATE] Write each example on the whiteboard and solve with the group.

11:45 - 12:15 | EXERCISE: Calculator Game

[SETUP] Give each participant a worksheet with 10 expressions.

[SAY] "Solve these WITHOUT a calculator. Just logic. We'll go through them together."

Worksheet:

1. $5 + 3 \times 2 = ?$ (Answer: 11 – teach precedence)
2. `"Hello" + " " + "World" = ?` (Answer: `"Hello World"`)
3. $10 > 5$ AND `true` = ? (Answer: `true`)
4. $[1,2] + [3,4] = ?$ (Answer: $[1,2,3,4]$)
5. $15 \% 4 = ?$ (Answer: 3 – remainder)
6. `"Apple".length > 3 = ?` (Answer: `true`)
7. $10 == "10" = ?$ (Answer: `true` – teach equality)
8. $10 === "10" = ?$ (Answer: `false` – strict equality)
9. NOT (`true` AND `false`) = ? (Answer: `true`)
10. $(5 + 5) * (2 + 3) = ?$ (Answer: 50 – parentheses matter!)

[GO THROUGH EACH ANSWER] Explain step-by-step. Emphasize:

- Multiplication before addition (precedence)
- Parentheses override precedence

- Text concatenation with `+`
- `==` vs `===` (value vs value AND type)

Afternoon Session Script

13:00 - 14:00 | EXERCISE: Spot the Type Error

[SAY] "Now for the tricky part. Sometimes we mix wrong types. Let's see if you can spot the errors. "

[PROJECT OR WRITE]

Expression 1: `"Hello" * 3`

→ Why `is` this wrong? (Can't multiply text!)

Expression 2: `5 + true`

→ Why `is` this wrong? (Can't add number `and` boolean!)

Expression 3: `[1,2] + 3`

→ Why `is` this wrong? (Can't add list `and` number!)

Expression 4: `10 / 0`

→ Why `is` this wrong? (Can't divide `by` zero – runtime `error`!)

Expression 5: `"5" + 3`

→ Surprise! This WORKS. `"5" + 3 = "53"`
(Text takes `over`, converts number `to` text)

[SAY] "So rule of thumb: **Don't mix types unless you're sure what happens.** "

14:00 - 15:00 | BUILD: Complete the Data Map

[SAY] "Now let's build a complete data map for a REAL scenario."

[WRITE ON WHITEBOARD]

Scenario: You're building a student management system.

[SAY] "Let's identify ALL the data types, variables, and operators we'd use."

Group Exercise (15 minutes):

- Divide into teams of 3-4
- Each team builds a data map on flip chart paper:

- i. List all data types needed
- ii. Name all variables
- iii. Write 5 operations (operators) we'd use

Team Presentations (10 minutes each):

- Each team presents their data map
- Facilitator adds feedback

[FACILITATOR CHEAT SHEET - What they should have]

Data Type	Variables	Operations
Number	age , grade , score	+ , - , * , / , >
Text	name , course , email	+ , .length
Boolean	isPassing , isEnrolled	AND , OR , NOT
List	scores , courses	.length , +
Nothing	missingData	is null?

15:00 - 15:30 | DAY 1 WRAP-UP

[SAY] "Let's recap Day 1."

[WRITE ON WHITEBOARD - Have participants fill in]

1. Data **Types**: _____, _____, _____, _____, _____
2. Variables = **Name** that points **to** a _____
3. FP **Rule**: Variables are _____ (never change)
4. Operators = _____ we can **perform on** data
5. **Type** Error = Mixing _____ **types**

Answers:

1. Numbers, Text, Booleans, Lists, Nothing
2. Value
3. Immutable
4. Actions
5. Different

[SAY] "Before we finish, one question: **What's one thing that surprised you today?** "

(Collect answers - write on whiteboard)

[CLOSING STATEMENT] "Today we learned WHAT data is and WHAT we can do with it. Tomorrow we'll learn HOW to ASK questions (expressions), GIVE commands (statements), and MAKE DECISIONS (conditions). You'll be writing complete logic by this time tomorrow."

[HOMEWORK] "Tonight, think about a process you do every day (making coffee, commuting, etc.). Write down:

1. What data types are involved?
2. What variables would you name?
3. What operations do you perform?"

Day 1 Facilitator Cheat Sheet

Mistake	How to Fix
Confusing "123" (text) with 123 (number)	"Quotes = text. No quotes = number."
Trying to divide text	"You can't divide 'hello' by 3. Only numbers."
Erasing variables	"Don't erase! Create new cards. History matters."
Mixing == and ===	"== checks value. === checks value AND type."
Not knowing precedence	"Parentheses force order. Use them if unsure."

Day 2 Facilitator Guide

Expressions, Statements & Conditions

Session Plan Overview

Time	Activity	Duration
09:00-09:15	Day 1 Recap	15 min
09:15-09:45	Expressions Theory	30 min
09:45-10:15	Statements Theory	30 min
10:15-10:45	EXERCISE: Expression or Statement	30 min
10:45-11:15	Role-Play: Two Cashiers	30 min
11:15-11:45	Conditions Theory	30 min

Time	Activity	Duration
11:45-12:15	EXERCISE: Build Decision Trees	30 min
12:15-13:00	LUNCH	45 min
13:00-14:00	EXERCISE: Spot the Statement	60 min
14:00-15:00	BUILD: Complete Decision System	60 min
15:00-15:30	Day 2 Wrap-Up	30 min

Morning Session Script

09:00 - 09:15 | Day 1 Recap

[SAY] "Quick recap from yesterday. **Name one data type.** "

(Call on random participants)

"What does immutable mean? "

(Answer: Cannot be changed)

"What's a type error? "

(Answer: Mixing wrong types)

"Excellent. Now we BUILD on that."

09:15 - 09:45 | Expressions Theory

[SAY] "Now we enter the CORE of programming. **Everything in programming is about asking questions and getting answers.** "

[WRITE ON WHITEBOARD]

EXPRESSION = A question that produces a VALUE.

Examples:

"What is `2 + 2`?" → 4 (expression)

"What is `'Hello' + ' World'`?" → "Hello World" (expression)

"Is `10 > 5`?" → true (expression)

[SAY] "Notice: **Every expression gives you something back.** You ask a question, you get an answer. No side effects. Nothing else happens."

[DEMONSTRATE] Act out an expression:

[FACILITATOR ACTION] Hold up a card with "5 + 3". "This is an expression. If I ask 'What is this?', you say '8'. Nothing else happens. The room doesn't change. The time doesn't change. Only the answer matters."

[SAY] "Now hold up a card with "print 'Hello'". "This is NOT an expression. Why? Because if I ask 'What is this?', you can't give me a value. Something HAPPENS. Something changes (the screen shows 'Hello')."

[WRITE ON WHITEBOARD]

Expression = ASKS a question → Produces VALUE
Statement = GIVES a command → Performs ACTION

09:45 - 10:15 | Statements Theory

[SAY] "Now let's talk about statements. Statements are commands. They don't ask questions. They TELL the computer to do something."

[WRITE ON WHITEBOARD]

STATEMENTS = Commands that perform actions.

Examples:

- "Print 'Hello' to the screen"
- "Save this file"
- "Send an email"
- "Change the font color"

[SAY] "Statements are dangerous in functional programming. Why? Because they CHANGE things outside the program. They have **SIDE EFFECTS**."

[DRAMATIC PAUSE]

"In FP, we try to have as FEW statements as possible. Ideally, just ONE at the very end."

[WRITE ON WHITEBOARD]

FP Rule: All logic = EXPRESSIONS
Only the final output = STATEMENTS

[SAY] "Think of it like cooking. All the prep work (chopping, mixing, measuring) is like expressions. You get outputs (chopped onions, mixed batter). The actual COOKING (putting it in the oven) is a statement. It changes the world (food is now hot)."

10:15 - 10:45 | EXERCISE: Expression or Statement?

[SAY] "Let's practice. I'll call out a phrase. You hold up:

- **GREEN CARD** if it's an **EXPRESSION** (produces a value)
- **RED CARD** if it's a **STATEMENT** (performs an action)"

[DISTRIBUTE] Give everyone a green and red card.

[CALL OUT - One at a time]

Phrase	Answer	Explanation
"Add 5 and 3"	✅ EXPRESSION	Produces 8
"Print 'Hello' to the screen"	❌ STATEMENT	Action, no value
"What is the capital of France?"	✅ EXPRESSION	Produces "Paris"
"Write your name on the board"	❌ STATEMENT	Action, no value
"Calculate the area of a circle"	✅ EXPRESSION	Produces a number
"Save this file"	❌ STATEMENT	Action, no value
"Is 10 greater than 5?"	✅ EXPRESSION	Produces true
"Send an email"	❌ STATEMENT	Action, no value
"Find the length of 'Hello'"	✅ EXPRESSION	Produces 5
"Turn on the light"	❌ STATEMENT	Action, no value

[DEBRIEF]

- **Why is 'Calculate the area' an expression?** → It gives you a number back.
- **Why is 'Print' a statement?** → It does something, gives nothing back.

[SAY] "The key insight: **Expressions give you something you can use later. Statements just DO things and move on.** "

10:45 - 11:15 | Role-Play: Two Cashiers

[SETUP] Choose 2 volunteers:

- **Cashier A** (Expression Style)
- **Cashier B** (Statement Style)

[SAY] "We're going to role-play two cashiers. Watch carefully."

Cashier A - Expression Style: [FACILITATOR SCRIPT] "Cashier A, a customer gives you \$20 and asks for the total of \$15 items. What do you do?"

Cashier A (Expression): "I calculate the total. \$15. I give the customer a receipt with the total. I do NOT touch the cash drawer. I just compute."

[SAY] "What did Cashier A do? Did they change anything?"

(Answer: No, just calculated)

Cashier B - Statement Style: [FACILITATOR SCRIPT] "Cashier B, a customer gives you \$20 and asks for the total of \$15 items. What do you do?"

Cashier B (Statement): "I open the drawer. I take \$15. I put it in the drawer. I give \$5 change. I update the drawer balance."

[SAY] "What did Cashier B do? Did they change anything?"

(Answer: Yes, drawer balance changed)

[SAY] "Now, question for everyone: **Which cashier is easier to test if something goes wrong?** "

(Answer: Cashier A - because you can just check the calculation)

[SAY] "Exactly. Cashier A is **predictable**. Cashier B has **side effects**. In FP, we want to be Cashier A as much as possible."

11:15 - 11:45 | Conditions Theory

[SAY] "Now let's add DECISIONS. Computers aren't smart - they just follow rules. But we can make them APPEAR smart by using conditions."

[WRITE ON WHITEBOARD]

CONDITION = A decision point.

Structure:

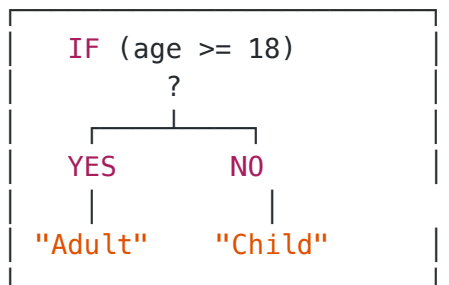
```
IF (something is true) THEN do this  
ELSE do something else
```

Example:

```
IF (age >= 18) THEN print "Adult"  
ELSE print "Child"
```

[SAY] "Think of a fork in the road. If it's raining, you take the umbrella path. If not, you take the sunglasses path."

[DRAW ON WHITEBOARD]



[SAY] "Conditions use BOOLEANS. Remember? True or false. The condition must evaluate to true or false."

[WRITE EXAMPLES]

```
IF (score >= 60) THEN print "Pass"
ELSE print "Fail"
```

```
IF (isRaining AND haveUmbrella) THEN goOutside
ELSE stayInside
```

```
IF (temperature > 30) THEN turnOnAC
ELSE IF (temperature < 10) THEN turnOnHeater
ELSE doNothing
```

11:45 - 12:15 | EXERCISE: Build Decision Trees

[SAY] "Your turn. I'll give you scenarios. You draw the decision tree on your paper."

[DISTRIBUTE] Give each participant blank paper.

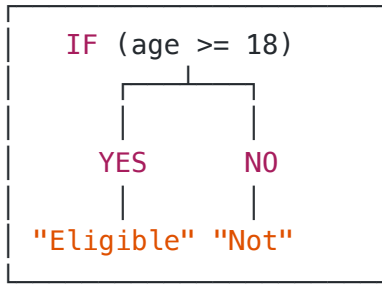
[PROJECT/WRITE SCENARIOS - One at a time]

Scenario 1:

"A program that checks if someone can vote. If age >= 18, print 'Eligible'. Otherwise, print 'Not eligible'."

Give them 2 minutes to draw.

[FACILITATOR SOLUTION]

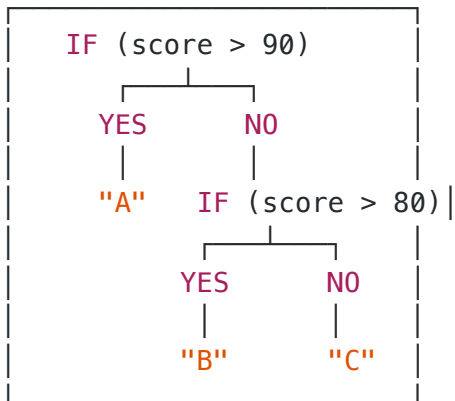


Scenario 2:

"A grading system. If score > 90, grade = A. If score > 80, grade = B. Otherwise, grade = C."

Give them 3 minutes.

[FACILITATOR SOLUTION]

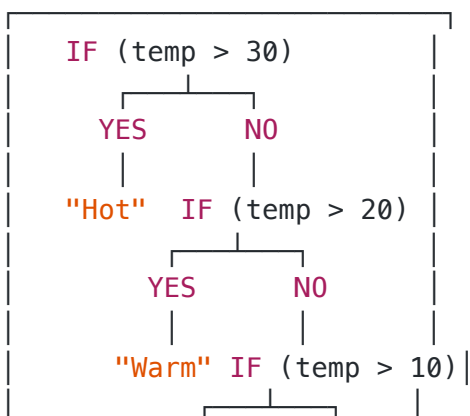


Scenario 3:

"A weather app. If temperature > 30, print 'Hot'. Else if temperature > 20, print 'Warm'. Else if temperature > 10, print 'Cool'. Else print 'Cold'."

Give them 4 minutes.

[FACILITATOR SOLUTION]





Afternoon Session Script

13:00 - 14:00 | EXERCISE: Spot the Statement

[SAY] "Now let's practice identifying statements vs expressions inside a 'recipe'."

[PROJECT/WRITE]

Recipe: Calculate total price with discount.

1. Set `total` = 0
2. For each item in cart:
3. Add item.price to `total`
4. Calculate discount = `total` * 0.1
5. Update `total` = `total` - discount
6. Print `total`

[SAY] "Let's identify which lines are STATEMENTS and which are EXPRESSIONS."

Group Discussion (10 minutes):

- Line 1: Set total = 0 → **STATEMENT** (changes total)
- Line 2: For each item → **STATEMENT** (loop command)
- Line 3: Add price to total → **STATEMENT** (changes total)
- Line 4: Calculate discount = total * 0.1 → **EXPRESSION** (produces a value)
- Line 5: Update total → **STATEMENT** (changes total)
- Line 6: Print total → **STATEMENT** (action)

[SAY] "Now let's REWRITE this in FP style. **Only the last line should be a statement.** "

[FACILITATOR SOLUTION]

FP Version:

1. `total` = 0 (**variable** - immutable)
2. For each item: (**loop** - but we'll replace **with** a LIST OPERATION)
 Actually, let's use expressions only:
3. `subtotal` = `total` of **all** item.prices (expression)
4. `discount` = `subtotal` * 0.1 (expression)

5. `finalTotal = subtotal - discount` (expression)
6. `Print finalTotal` (statement)

[SAY] "Notice: Lines 3, 4, 5 are now expressions. They produce new values. They don't change anything. Only line 6 is a statement."

14:00 - 15:00 | BUILD: Complete Decision System

[SAY] "Now let's build a COMPLETE system using everything we've learned."

[WRITE ON WHITEBOARD]

Scenario: A ticket pricing system **for** a cinema.

- Regular ticket: **\$10**
- Student ticket: **\$8** (must be 18+ **AND** have student ID)
- Child ticket: **\$5** (must be under 12)
- Senior ticket: **\$7** (must be 65+)

Group Exercise (30 minutes):

- Teams of 3-4
- Design the decision tree
- Identify all variables, expressions, and statements
- Draw the complete flow

[FACILITATOR SOLUTION - Show after group work]

Variables:

`age = 25`

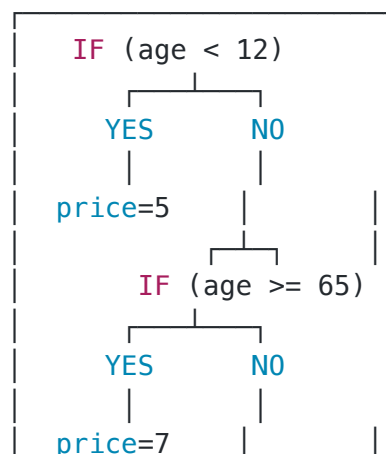
`isStudent = true`

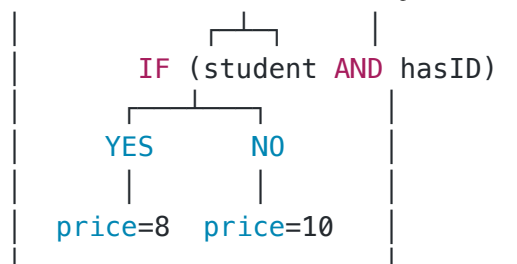
`hasStudentID = true`

`basePrice = 10`

`finalPrice = ?`

Decision Tree:





Expressions:

isChild = age < 12

isSenior = age >= 65

isQualifiedStudent = isStudent AND hasStudentID

Statement (ONLY ONE AT THE END):

Print finalPrice

15:00 - 15:30 | DAY 2 WRAP-UP

[SAY] "Let's recap Day 2."

[WRITE ON WHITEBOARD]

1. Expression = ____ that produces a ____
2. Statement = ____ that performs an ____
3. FP Rule: All logic = ____; Only output = ____
4. Condition = ____ based on boolean
5. Decision Tree = Visual representation of ____

Answers:

1. Question, Value
2. Command, Action
3. Expressions, Statement
4. Decision
5. Conditions

[SAY] "One question: **What's the biggest difference between an expression and a statement?** "

(Answer: Expression gives you a value; statement does an action)

[CLOSING STATEMENT] "Today we learned how to ASK questions (expressions), GIVE commands (statements), and MAKE DECISIONS (conditions). Tomorrow we'll learn how to REPEAT things (loops) and ORGANIZE complex data (lists and records)."

[HOMEWORK] "Think about a decision you make daily. Draw the decision tree for:

- 'What should I wear today?' (based on weather, occasion, etc.)
- Or 'What should I eat for lunch?'"

Day 2 Facilitator Cheat Sheet

Mistake	How to Fix
Calling "Print" an expression	"Does it give you a value? No. It does an action."
Forgetting ELSE in conditions	"What happens if the condition is false? Always handle both paths."
Nesting too many conditions	"Simplify! Use AND/OR to combine conditions."
Mixing expressions and statements	"Expressions give values. Statements DO things."
Overusing statements	"FP Rule: Push statements to the edge. Use expressions everywhere else."

Day 3 Facilitator Guide

Loops, Lists & Data Structures

Session Plan Overview

Time	Activity	Duration
09:00-09:15	Day 2 Recap	15 min
09:15-09:45	Loops Theory	30 min
09:45-10:15	Lists Theory	30 min
10:15-10:45	Records Theory	30 min
10:45-11:15	EXERCISE: Manual Loop Simulation	30 min
11:15-11:45	EXERCISE: While Loop Role-Play	30 min
11:45-12:15	EXERCISE: Build a Shopping Cart	30 min
12:15-13:00	LUNCH	45 min
13:00-14:00	EXERCISE: Design a Student Record	60 min

Time	Activity	Duration
14:00-15:00	BUILD: Complete Data System	60 min
15:00-15:30	Day 3 Wrap-Up	30 min

Morning Session Script

09:00 - 09:15 | Day 2 Recap

[SAY] "Quick recap. **What's an expression?** "

(Answer: Produces a value)

"**What's a statement?** "

(Answer: Performs an action)

"**What's a condition?** "

(Answer: Decision based on true/false)

"Excellent. Today we learn about REPETITION and ORGANIZATION."

09:15 - 09:45 | Loops Theory

[SAY] "Now we come to one of the most powerful concepts: **DOING THE SAME THING MANY TIMES WITHOUT WRITING IT OUT.** "

[WRITE ON WHITEBOARD]

LOOP = A way **to repeat** instructions.

Three types **of** loops:

1. **FOR EACH** item **in** a list → **do** something **to** every item
2. **WHILE** condition **is true** → keep doing **until** condition changes
3. **REPEAT** N times → **do** exactly N times

[SAY] "Think about washing dishes. You don't wash one plate, dry it, and stop. You repeat for every plate. That's a loop."

[DEMONSTRATE]

[FACILITATOR ACTION] Hold up a stack of 5 plates.

"Washing dishes:"

1. Take a plate
2. Wash it
3. Dry it
4. Put it away
5. Go back to step 1 UNTIL no plates left

"This is a loop. Specifically, a 'FOR EACH plate in stack' loop."

09:45 - 10:15 | Lists Theory

[SAY] "Before we can loop, we need something to loop OVER. That's where LISTS come in."

[WRITE ON WHITEBOARD]

LIST = An ordered collection of items.

Examples:

numbers = [10, 20, 30, 40, 50]

names = ["Alice", "Bob", "Charlie"]

mixed = [1, "hello", true, [2,3]]

Operations on lists:

- Get length: numbers.length = 5
- Get first item: numbers[0] = 10
- Add item: numbers + [60] = [10,20,30,40,50,60]
- Check if contains: 20 IN numbers = true

[SAY] "Lists hold things in ORDER. Position 0 is first, position 1 is second, etc. This is important for loops."

[DRAW ON WHITEBOARD]

List: [10, 20, 30, 40, 50]

10	20	30	40	50
----	----	----	----	----

Pos: 0 1 2 3 4

[SAY] "Index 0, 1, 2, 3, 4. Always start at zero. This is called **zero-based indexing**."

10:15 - 10:45 | Records Theory

[SAY] "Lists are great for multiple items of the SAME type. But what if you have different types of data for ONE thing?"

[WRITE ON WHITEBOARD]

RECORD/Object = A collection of NAMED fields.

Example:

```
person = {
  name: "Alice",
  age: 25,
  city: "London",
  isStudent: true,
  scores: [85, 90, 78]
}
```

Access:

```
person.name → "Alice"
person.age → 25
person.scores → [85, 90, 78]
```

[SAY] "Think of a record as a filing cabinet. Each drawer has a label (name), and inside is a value."

[DRAW ON WHITEBOARD]

PERSON RECORD	
name:	"Alice"
age:	25
city:	"London"
isStudent:	true
scores:	[85, 90, 78]

[SAY] "Lists and records together are called **DATA STRUCTURES**. They organize your data so you can work with it efficiently."

10:45 - 11:15 | EXERCISE: Manual Loop Simulation

[SAY] "Let's physically simulate a loop. Everyone stand up."

[FACILITATOR ACTION] Give 10 participants a card with a name on it. Have them stand in a line.

[SAY] "These are our names: ['Alice', 'Bob', 'Charlie', 'David', 'Eve', 'Frank', 'Grace', 'Henry', 'Ivy', 'Jack']"

"Here's our instruction: **For each name in the list, say 'Hello, [name]!' "**

[DEMONSTRATE] "I'll go first: Hello, Alice!"

[Participant 2] "Hello, Bob!" [Participant 3] "Hello, Charlie!" ...

[SAY] "Notice: We repeated the SAME instruction for each person. That's a loop."

[WRITE THE LOOP]

```
FOR each name in names:
    print "Hello, " + name
```

[SAY] "Now let's do a different loop. For each name, add '!' to the end. "

[Participant 1] "Alice!" [Participant 2] "Bob!" ...

[SAY] "See? Same loop structure, different action inside."

11:15 - 11:45 | EXERCISE: While Loop Role-Play

[SAY] "Now let's simulate a WHILE loop. This is a loop that continues UNTIL a condition changes."

[SETUP] Choose 1 person to be the "Guard". Give them a secret password (e.g., "1234" on a card).

[SAY] "The guard will ask for a password. Everyone else will guess. The loop continues WHILE the guess is wrong. "

Simulation:

```
WHILE (guess != "1234"):
    Ask for password
    Get guess
    If guess is correct: exit loop
    Else: repeat
```

[ACT OUT]

- Participant 1: "Password is 'apple'" → Guard: "Wrong"
- Participant 2: "Password is '123'" → Guard: "Wrong"
- Participant 3: "Password is '1234'" → Guard: "Correct!"

[SAY] "Notice: The loop stops when the condition (guess != '1234') becomes FALSE."

[WRITE ON WHITEBOARD]

```
WHILE condition is true:
    DO something
    Check condition again
```

Stop when condition becomes false.

11:45 - 12:15 | EXERCISE: Build a Shopping Cart

[SAY] "Now let's apply loops and lists to a real scenario."

[WRITE ON WHITEBOARD]

Scenario: Shopping Cart

Products:

- Apple: \$1.00
- Banana: \$2.00
- Cherry: \$3.00
- Date: \$4.00
- Elderberry: \$5.00

Cart: ['apple', 'banana', 'cherry']

[SAY] "Task: Calculate the total price of items in the cart."

Step-by-step with group:

1. Start with total = 0
2. For each item in cart:
3. Find the price of the item
4. Add price to total
5. After the loop, print total

[FACILITATOR WALKTHROUGH]

"Let's do this manually:"

total = 0

Loop 1: item = 'apple' → price = 1.00 → total = 1.00

Loop 2: item = 'banana' → price = 2.00 → total = 3.00

Loop 3: item = 'cherry' → price = 3.00 → total = 6.00

Final total = \$6.00

[SAY] "Now your turn. Add 'date' and 'elderberry' to the cart. Calculate the new total."

[GIVE 2 MINUTES]

"Answer: \$15.00 (6 + 4 + 5)"

Afternoon Session Script

13:00 - 14:00 | EXERCISE: Design a Student Record

[SAY] "Now let's design a COMPLETE data structure for a student."

[WRITE ON WHITEBOARD]

Student Record:

- Name (text)
- Age (number)
- City (text)
- Subjects (list of text)
- Grades (list of numbers)

[SAY] "Draw this record on your paper. Include sample data."

[FACILITATOR SOLUTION]

```
student = {  
  name: "Alice Johnson",  
  age: 20,  
  city: "London",  
  subjects: ["Math", "Science", "History", "English"],  
  grades: [85, 92, 78, 88]  
}
```

[SAY] "Now calculate the average grade."

Step-by-step:

1. total = 0
2. FOR each grade in grades:
3. total = total + grade
4. average = total / length of grades
5. Print average

Manual calculation:

```
grades = [85, 92, 78, 88]  
total = 85 + 92 + 78 + 88 = 343  
average = 343 / 4 = 85.75
```

14:00 - 15:00 | BUILD: Complete Data System

[SAY] "Now let's build a COMPLETE system using loops, lists, and records."

[WRITE ON WHITEBOARD]

Scenario: Class Management System

You have 3 students with records.

For each student:

- Calculate their average grade
- Determine if they pass (average ≥ 60)
- Count how many students pass
- Print the class average

Group Exercise (30 minutes):

Teams of 3-4. Design the complete system.

[FACILITATOR SOLUTION]

Data:

```
students = [  
  { name: "Alice", grades: [85, 90, 78] },  
  { name: "Bob", grades: [55, 60, 65] },  
  { name: "Charlie", grades: [92, 88, 94] }  
]
```

Algorithm:

1. passCount = 0
2. classTotal = 0
3. FOR each student in students:
4. studentTotal = 0
5. FOR each grade in student.grades:
6. studentTotal = studentTotal + grade
7. studentAvg = studentTotal / length of student.grades
8. IF studentAvg ≥ 60 :
9. passCount = passCount + 1
10. Print student.name + " passes with " + studentAvg
11. ELSE:
12. Print student.name + " fails with " + studentAvg
13. classTotal = classTotal + studentAvg
14. classAvg = classTotal / length of students
15. Print "Class average: " + classAvg
16. Print "Passing students: " + passCount

Output:

Alice passes with 84.33
Bob passes with 60.00
Charlie passes with 91.33
Class average: 78.55
Passing students: 3

15:00 - 15:30 | DAY 3 WRAP-UP

[SAY] "Let's recap Day 3."

[WRITE ON WHITEBOARD]

1. Loop = Repeating instructions
2. Three loops: , ,
3. List = collection of items
4. Record = fields with labels
5. Data Structures = + combined

Answers:

- 1.
2. FOR EACH, WHILE, REPEAT N
3. Ordered
4. Named
5. Lists, Records

[SAY] "One question: What's the difference between a list and a record? "

(Answer: List is ordered with numbers; record has named fields)

[CLOSING STATEMENT] "Today we learned how to REPEAT (loops), ORGANIZE (lists), and GROUP (records). Tomorrow we'll learn how to PACKAGE logic into FUNCTIONS and control SCOPE."

[HOMEWORK] "Think about your favorite app (Instagram, Spotify, etc.). What lists and records do you think it uses?"

Day 3 Facilitator Cheat Sheet

Mistake	How to Fix
Off-by-one errors (starting at 1)	"Lists start at position 0. Always."
Infinite loops	"Make sure the condition CHANGES each time."

Mistake	How to Fix
Confusing list and record	"List = ordered. Record = labeled."
Forgetting to update loop variable	"Each loop must change something."
Nested loops confusion	"Outer loop = one thing. Inner loop = details."

Day 4 Facilitator Guide

Functions, Scope & Errors

Session Plan Overview

Time	Activity	Duration
09:00-09:15	Day 3 Recap	15 min
09:15-09:45	Functions Theory	30 min
09:45-10:15	Function Examples	30 min
10:15-10:45	Scope Theory	30 min
10:45-11:15	EXERCISE: Scope Game	30 min
11:15-11:45	Errors Theory	30 min
11:45-12:15	EXERCISE: Fix Broken Recipes	30 min
12:15-13:00	LUNCH	45 min
13:00-14:00	EXERCISE: Write Functions	60 min
14:00-15:00	BUILD: Complete Function System	60 min
15:00-15:30	Day 4 Wrap-Up	30 min

Morning Session Script

09:00 - 09:15 | Day 3 Recap

[SAY] "Quick recap. What's a loop? "

(Answer: Repeating instructions)

"What's a list? "

(Answer: Ordered collection)

"What's a record? "

(Answer: Named fields)

"Excellent. Today we learn how to PACKAGE logic."

09:15 - 09:45 | Functions Theory

[SAY] "Imagine you have a recipe for chocolate cake. Every time you want cake, you don't re-invent the recipe. You just say 'follow the cake recipe.' That's a FUNCTION."

[WRITE ON WHITEBOARD]

FUNCTION = A **named** block of logic.

Parts:

1. **NAME** (what **do** you **call** it?)
2. **INPUT** (what **do** you give it?)
3. **PROCESSING** (what does it **do**?)
4. **OUTPUT** (what does it give back?)

Example:

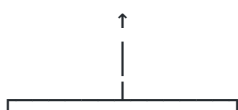
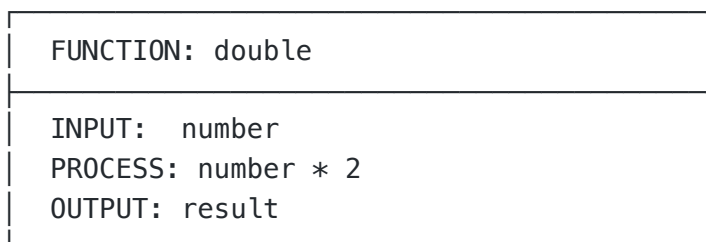
```
FUNCTION double(number):  
    return number * 2
```

Calling the **function**:

```
double(5) → 10  
double(10) → 20  
double(0.5) → 1
```

[SAY] "Functions are REUSABLE. You write once, use everywhere."

[DRAW ON WHITEBOARD]



double(5)	→	10
double(10)	→	20
double(3)	→	6

09:45 - 10:15 | Function Examples

[SAY] "Let's write some functions together."

[WRITE ON WHITEBOARD - WITH GROUP]

Function 1: Greet Someone

```
FUNCTION greet(name):  
    return "Hello, " + name  
  
greet("Alice") → "Hello, Alice"  
greet("Bob") → "Hello, Bob"
```

Function 2: Calculate Circle Area

```
FUNCTION circleArea(radius):  
    return 3.14159 * radius * radius  
  
circleArea(5) → 78.54  
circleArea(10) → 314.16
```

Function 3: Check if Even

```
FUNCTION isEven(number):  
    return number % 2 == 0  
  
isEven(4) → true  
isEven(7) → false
```

Function 4: Convert Celsius to Fahrenheit

```
FUNCTION cToF(celsius):  
    return (celsius * 9/5) + 32  
  
cToF(0) → 32  
cToF(100) → 212  
cToF(25) → 77
```

[SAY] "Notice: Each function takes INPUT, does PROCESSING, and gives OUTPUT. Clean and simple."

10:15 - 10:45 | Scope Theory

[SAY] "Now for a tricky concept: SCOPE. **Where can you see a variable?** "

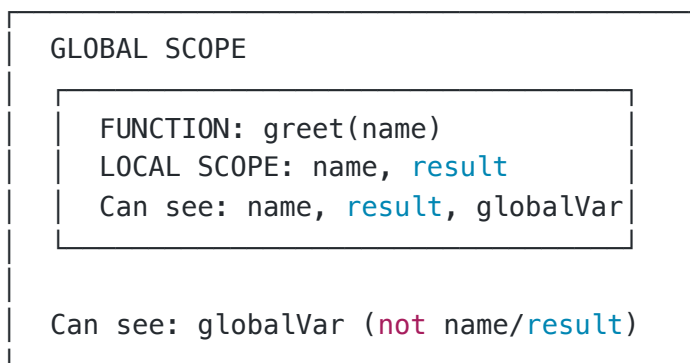
[WRITE ON WHITEBOARD]

SCOPE = The visibility of a variable.

Two types:

1. LOCAL = Inside a function (only visible there)
2. GLOBAL = Outside all functions (visible everywhere)

[DRAW ON WHITEBOARD]



[SAY] "Think of your house. The living room is GLOBAL (everyone can see it). Your bedroom is LOCAL (only you can see what's inside)."

[DEMONSTRATE]

[FACILITATOR ACTION] Set up 3 boxes on a table:

- **Global Box:** Contains "houseAddress" label
- **Bedroom Box:** Contains "pillow" label
- **Kitchen Box:** Contains "fridge" label

[SAY] "Can the Kitchen see the Bedroom's pillow?" (*No - different scope*)

"Can the Bedroom see the Kitchen's fridge?" (*No - different scope*)

"Can everyone see the house address?" (*Yes - global*)

"Same in programming: Functions can't see each other's local variables."

10:45 - 11:15 | EXERCISE: Scope Game

[SAY] "Let's play a game. I'll give you a scenario. Tell me what's visible."

[PROJECT/WRITE]

Scenario 1:

```
GLOBAL: city = "London"
```

```
FUNCTION getWeather():  
    LOCAL: temperature = 25  
    PRINT temperature  
    PRINT city
```

Question: What can `getWeather()` see? **Answer:** `temperature` (local) AND `city` (global)

Scenario 2:

```
FUNCTION printName():  
    LOCAL: name = "Alice"  
    PRINT name
```

```
FUNCTION printGreeting():  
    LOCAL: greeting = "Hello"  
    PRINT greeting  
    PRINT name ← ERROR: name is not visible here!
```

Question: Can `printGreeting()` see `name`? **Answer:** No! `name` is local to `printName()`.

11:15 - 11:45 | Errors Theory

[SAY] "Programs break. It's normal. The key is understanding WHY."

[WRITE ON WHITEBOARD]

Three types of errors:

1. SYNTAX **ERROR**: You wrote gibberish
2. RUNTIME **ERROR**: Valid but breaks **when** running
3. LOGIC **ERROR**: Runs but gives wrong answer

[SAY] "Think about a recipe:"

Error Type	Recipe Analogy	Programming Analogy
Syntax	"Bkat cake at 350" (spelling)	<code>5 + * 3</code> (invalid)
Runtime	"Add salt" but you have no salt	<code>10 / 0</code> (divide by zero)
Logic	"Bake at 350" but you use 180	Adding instead of subtracting

[DEMONSTRATE EACH]

Syntax Error:

```
5 + * 3 ← What's wrong?
```

(Answer: ** has no left operand*)

Runtime Error:

```
divide(10, 0) ← What happens?
```

(Answer: *Can't divide by zero*)

Logic Error:

```
FUNCTION rectangleArea(width, height):  
    return width + height ← WRONG! Should be width * height
```

[SAY] "Logic errors are the hardest. The program runs perfectly but gives the WRONG answer. That's why we test."

11:45 - 12:15 | EXERCISE: Fix the Broken Recipes

[SAY] "Let's fix these broken recipes."

[PROJECT/WRITE]

Error 1 - Syntax:

```
FUNCTION double(number)  
    return number * 2 ← What's missing?
```

Answer: Missing parentheses after FUNCTION name.

Fixed:

```
FUNCTION double(number):  
    return number * 2
```

Error 2 - Runtime:

```
FUNCTION average(scores):  
    total = 0  
    FOR each score in scores:  
        total = total + score  
    return total / 0    ← What's wrong?
```

Answer: Dividing by zero!

Fixed:

```
FUNCTION average(scores):  
    total = 0  
    FOR each score in scores:  
        total = total + score  
    count = length of scores  
    IF count == 0:  
        return 0  
    ELSE:  
        return total / count
```

Error 3 - Logic:

```
FUNCTION discountPrice(price):  
    return price - 10    ← WRONG: Should be 10% off
```

Answer: Should be price * 0.9

Fixed:

```
FUNCTION discountPrice(price):  
    return price * 0.9
```

Afternoon Session Script

13:00 - 14:00 | EXERCISE: Write Functions

[SAY] "Your turn. Write a 'recipe' (function) for each of these."

[PROJECT/WRITE]

1. `square(number)` → returns `number × number`
2. `isAdult(age)` → returns `true` if `age >= 18`
3. `max(a, b)` → returns the larger of `a` and `b`
4. `length(text)` → returns the number of characters
5. `addAll(list)` → returns sum of all numbers in list

Give 10 minutes.

[FACILITATOR SOLUTIONS]

1. **FUNCTION** `square(number):`
 `return number * number`
2. **FUNCTION** `isAdult(age):`
 `return age >= 18`
3. **FUNCTION** `max(a, b):`
 IF `a > b` THEN `return a`
 ELSE `return b`
4. **FUNCTION** `length(text):`
 `return text.countCharacters`
5. **FUNCTION** `addAll(list):`
 `total = 0`
 FOR each number in list:
 `total = total + number`
 `return total`

[SAY] "Notice: Function 5 uses a loop INSIDE a function. That's fine! Functions can contain any logic."

14:00 - 15:00 | BUILD: Complete Function System

[SAY] "Now let's build a COMPLETE system using functions."

[WRITE ON WHITEBOARD]

Scenario: E-commerce Checkout

Functions needed:

1. `calculateSubtotal(cart)` → sum of all item prices
2. `applyDiscount(subtotal, coupon)` → apply coupon discount

3. `addTax(total, taxRate)` → add tax
4. `calculateTotal(cart, coupon, taxRate)` → final price

Group Exercise (30 minutes):

Teams design all functions.

[FACILITATOR SOLUTION]

```
FUNCTION calculateSubtotal(cart):  
  total = 0  
  FOR each item in cart:  
    total = total + item.price  
  return total
```

```
FUNCTION applyDiscount(subtotal, coupon):  
  IF coupon == "SAVE10":  
    return subtotal * 0.9  
  ELSE IF coupon == "SAVE20":  
    return subtotal * 0.8  
  ELSE:  
    return subtotal
```

```
FUNCTION addTax(total, taxRate):  
  return total * (1 + taxRate)
```

```
FUNCTION calculateTotal(cart, coupon, taxRate):  
  subtotal = calculateSubtotal(cart)  
  discounted = applyDiscount(subtotal, coupon)  
  final = addTax(discounted, taxRate)  
  return final
```

Testing:

`cart = [{name: "Book", price: 20}, {name: "Pen", price: 5}]`

`calculateTotal(cart, "SAVE10", 0.2) → ?`

Subtotal: 25

Discount: $25 * 0.9 = 22.5$

Tax: $22.5 * 1.2 = 27$

Final: \$27

15:00 - 15:30 | DAY 4 WRAP-UP

[SAY] "Let's recap Day 4."

[WRITE ON WHITEBOARD]

1. Function = block of logic
2. Function parts: , , ,

3. Scope = ____ of variables
4. Local = visible ____ function
5. Global = visible ____ functions
6. Three errors: ____, ____, ____

Answers:

1. Named
2. Name, Input, Processing, Output
3. Visibility
4. Inside
5. Outside all
6. Syntax, Runtime, Logic

[SAY] "One question: **What's the hardest type of error to find?** "

(Answer: Logic error - because it runs but gives wrong answer)

[CLOSING STATEMENT] "Today we learned how to PACKAGE logic (functions), CONTROL visibility (scope), and HANDLE problems (errors). Tomorrow we'll put EVERYTHING together to design a complete program."

[HOMEWORK] "Design a function for:

- 'Calculate the area of a triangle'
- 'Check if a number is prime'
- 'Find the largest number in a list''

Day 4 Facilitator Cheat Sheet

Mistake	How to Fix
Forgetting function parameters	"Every input must be listed as a parameter."
Trying to use local variables outside	"Local means LOCAL. It doesn't exist outside."
Mixing up return vs print	"Return gives value back; print just shows on screen."
Not handling edge cases	"What if the list is empty? What if input is negative?"
Logic errors	"Test with known values. If input 5 gives 10, good. If 5 gives 12, check."

Day 5 Facilitator Guide

Input/Output, Algorithms & Complete Design

Session Plan Overview

Time	Activity	Duration
09:00-09:15	Day 4 Recap	15 min
09:15-09:45	Input/Output Theory	30 min
09:45-10:15	EXERCISE: Waiter Simulation	30 min
10:15-10:45	Algorithms Theory	30 min
10:45-11:15	EXERCISE: Algorithm Steps	30 min
11:15-11:45	Complete Program Structure	30 min
11:45-12:15	EXERCISE: Design Complete Program	30 min
12:15-13:00	LUNCH	45 min
13:00-14:00	FINAL PROJECT: Pizza Shop	60 min
14:00-15:00	Project Work & Presentations	60 min
15:00-15:30	Final Wrap-Up & Certificates	30 min

Morning Session Script

09:00 - 09:15 | Day 4 Recap

[SAY] "Quick recap. **What's a function?** "

(Answer: Named block of logic)

"What's scope? "

(Answer: Visibility of variables)

"What are the three error types? "

(Answer: Syntax, Runtime, Logic)

"Excellent. Today we put it ALL together."

09:15 - 09:45 | Input/Output Theory

[SAY] "Programs exist in the real world. They need to RECEIVE data (input) and SEND data (output)."

[WRITE ON WHITEBOARD]

INPUT = Data coming into the program

OUTPUT = Data leaving the program

Sources of INPUT:

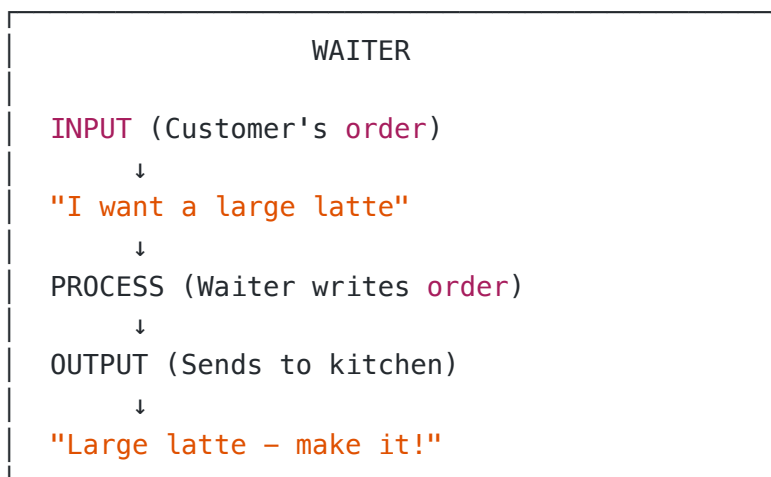
- User typing on keyboard
- Clicking a button
- Reading a file
- Getting data **from** internet (API)

Destinations **for** OUTPUT:

- Showing on screen
- Printing on paper
- Writing **to** a file
- Sending **to** internet (API)

[SAY] "Think of a waiter at a restaurant."

[DRAW ON WHITEBOARD]



[SAY] "In FP, we keep INPUT and OUTPUT at the EDGES. All the logic in the middle is PURE (no side effects)."

09:45 - 10:15 | EXERCISE: Waiter Simulation

[SETUP]

- 1 person = Customer
- 1 person = Waiter
- 1 person = Kitchen

[SAY] "Let's simulate a complete I/O flow."

Script:

Customer (Input): "I want a large latte with extra shot."

Waiter (Process):

- Takes order (input)
- Writes order on card
- Passes to kitchen

Kitchen (Output):

- Reads order
- Makes coffee
- Prints receipt: "Large Latte + Extra Shot"

[SAY] "Notice: The Waiter didn't CHANGE anything. They just PASSED data along. That's functional."

10:15 - 10:45 | Algorithms Theory

[SAY] "Now we learn about ALGORITHMS - the step-by-step instructions to solve problems."

[WRITE ON WHITEBOARD]

ALGORITHM = A sequence of steps to solve a problem.

Every program is an algorithm.

[SAY] "Three classic algorithms we'll learn today:"

1. Find the Maximum:

List: [3, 7, 2, 9, 1]

Algorithm:

1. Look at first number → call it "max"
2. Compare with next number
3. If next number is larger, update "max"
4. Repeat until end of list

5. Output "max"

Manual:

[3, 7, 2, 9, 1]

max = 3

Compare 3 and 7 → 7 is larger → max = 7

Compare 7 and 2 → 7 is larger → max = 7

Compare 7 and 9 → 9 is larger → max = 9

Compare 9 and 1 → 9 is larger → max = 9

Output: 9

2. Search for an Item:

List: ["apple", "banana", "cherry"]

Search for: "banana"

Algorithm:

1. Look at first item

2. If it matches, return "found"

3. If not, look at next

4. If end of list, return "not found"

Manual:

Check "apple" → no match

Check "banana" → match → found!

3. Sort a List (Bubble Sort - Simplified):

List: [5, 2, 8, 1, 3]

Algorithm:

1. Compare first two numbers

2. If first > second, swap them

3. Move to next pair

4. Repeat until all numbers are in order

Manual (simplified):

[5, 2, 8, 1, 3]

Compare 5 and 2 → swap → [2, 5, 8, 1, 3]

Compare 5 and 8 → ok

Compare 8 and 1 → swap → [2, 5, 1, 8, 3]

Compare 8 and 3 → swap → [2, 5, 1, 3, 8]

... continue until sorted → [1, 2, 3, 5, 8]

10:45 - 11:15 | EXERCISE: Algorithm Steps

[SAY] "Your turn. Write the algorithm for these problems."

Problem 1: Find the youngest person in a room.

[FACILITATOR SOLUTION]

Algorithm: Find Youngest

1. Look at first person → call them "youngest"
2. Look at next person
3. If next person is younger than "youngest":
4. Update "youngest" = this person
5. Repeat from step 2 until no more people
6. Output "youngest"

Problem 2: Check if a list contains the number 10.

[FACILITATOR SOLUTION]

Algorithm: Find Number 10

1. For each number in list:
2. If number == 10:
3. Return true (found)
4. After checking all numbers:
5. Return false (not found)

Problem 3: Calculate the average of a list of numbers.

[FACILITATOR SOLUTION]

Algorithm: Calculate Average

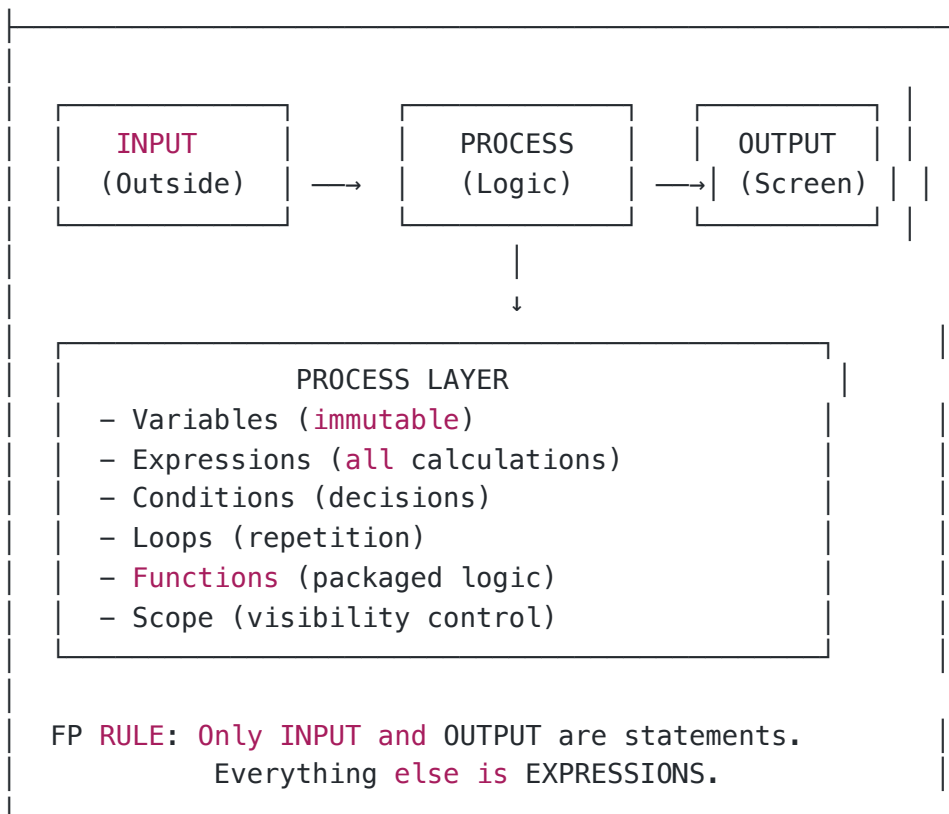
1. total = 0
2. For each number in list:
3. total = total + number
4. count = length of list
5. average = total / count
6. Output average

11:15 - 11:45 | Complete Program Structure

[SAY] "Now we put EVERYTHING together. A complete program follows this structure:"

[WRITE ON WHITEBOARD - Permanent]

COMPLETE PROGRAM



11:45 - 12:15 | EXERCISE: Design a Complete Program

[SAY] "Let's design a complete program from scratch."

[WRITE ON WHITEBOARD]

Scenario: Student Grade Calculator

Input: List of student scores

Process:

- Calculate **average**
- Determine **if** each student passes
- Count passing students

Output: Report **with** grades **and** pass/**fail** status

Group Design (20 minutes):

Teams design the complete program.

[FACILITATOR SOLUTION]

DATA TYPES:

- Number: scores, average
- Boolean: **isPassing**
- List: scores
- Text: student name, grade letter

VARIABLES:

- scores = [85, 90, 78, 55, 92]
- passingScore = 60
- studentName = "Alice"

EXPRESSIONS:

- average = sum(scores) / length(scores)
- isPassing = average >= passingScore
- gradeLetter = IF average > 90 "A" ELSE IF > 80 "B" ...

STATEMENTS:

- ONLY ONE: Print the final report

CONDITIONS:

- IF average >= 60 THEN "Pass" ELSE "Fail"
- IF average > 90 THEN "A" ...

LOOPS:

- FOR each student in students: calculate their average

FUNCTIONS:

- FUNCTION calculateAverage(scores) → returns average
- FUNCTION determineGrade(average) → returns letter
- FUNCTION isPassing(average) → returns true/false
- FUNCTION generateReport(student) → returns report text

SCOPE:

- scores is local to calculateAverage
- passingScore is global (constant)

INPUT:

- List of student records (from file or user)

OUTPUT:

- Report printed to screen

ALGORITHM:

1. Get list of students (input)
2. FOR each student:
3. avg = calculateAverage(student.scores)
4. grade = determineGrade(avg)
5. status = isPassing(avg)
6. Store result in report
7. Print report (output)

Afternoon Session Script

13:00 - 14:00 | FINAL PROJECT: Pizza Shop

[**SAY**] "Now for your final project. You'll design a complete program from scratch."

[WRITE ON WHITEBOARD]

PROJECT: Pizza Order System

SCENARIO:

You run a pizza shop. Customers can order custom pizzas.

PRICING:

- Base **pizza**: \$10
- Each **topping**: +\$2
- Large **size**: +\$5
- Family **size**: +\$10
- **Discount**: 10% **if** total > \$30
- **Tax**: 10% (applied after discount)

INPUT:

- **Size**: "small", "large", or "family"
- **Toppings**: list of toppings (e.g., ["pepperoni", "mushrooms", "olives"])
- **Coupon**: "SAVE10", "SAVE20", or none

OUTPUT:

- Order summary
- Subtotal
- Discount (**if** applied)
- Tax
- Final total

REQUIREMENTS:

1. Use ALL 5 data types
2. Use variables (immutable)
3. Use expressions
4. At least ONE condition
5. At least ONE loop
6. At least 3 functions
7. Proper scope
8. Input and output
9. Clear algorithm
10. No logic errors

14:00 - 15:00 | Project Work & Presentations

[SETUP]

- Teams of 3-4
- 45 minutes to design
- 15 minutes for presentations

Team Deliverables:

1. Data types identified
2. Variables listed
3. Expressions written
4. Conditions drawn
5. Loops defined
6. Functions designed
7. Scope documented
8. Input/output specified
9. Algorithm written
10. Error handling considered

[FACILITATOR SOLUTION - Reference]

DATA TYPES:

- Number: price, total, discount, tax
- Text: size, coupon, topping name
- Boolean: hasDiscount
- List: toppings
- **Nothing**: null (if no coupon)

VARIABLES (IMMUTABLE):

- basePrice = 10
- toppingPrice = 2
- sizeSurcharge = {small: 0, large: 5, family: 10}
- taxRate = 0.1
- discountRate = 0.1

EXPRESSIONS:

- toppingsTotal = length(toppings) * toppingPrice
- subtotal = basePrice + toppingsTotal + sizeSurcharge[size]
- hasDiscount = subtotal > 30
- discountAmount = **IF** hasDiscount **THEN** subtotal * discountRate **ELSE** 0
- discountedTotal = subtotal - discountAmount
- finalTotal = discountedTotal * (1 + taxRate)

CONDITIONS:

- **IF** size == "large" **THEN** add 5
- **IF** size == "family" **THEN** add 10
- **IF** total > 30 **THEN** apply 10% discount
- **IF** coupon == "SAVE10" **THEN** apply extra 10%
- **IF** coupon == "SAVE20" **THEN** apply extra 20%

LOOPS:

- **FOR** each topping **in** toppings:
 add price **to** total

FUNCTIONS:

1. FUNCTION calculateSubtotal(size, toppings):
 base = 10
 toppingCost = length(toppings) * 2
 sizeCost = getSizeCost(size)
 return base + toppingCost + sizeCost
2. FUNCTION getSizeCost(size):
 IF size == "large" THEN return 5
 IF size == "family" THEN return 10
 ELSE return 0
3. FUNCTION applyDiscount(total):
 IF total > 30 THEN return total * 0.9
 ELSE return total
4. FUNCTION applyCoupon(total, coupon):
 IF coupon == "SAVE10" THEN return total * 0.9
 IF coupon == "SAVE20" THEN return total * 0.8
 ELSE return total
5. FUNCTION calculateFinal(size, toppings, coupon):
 subtotal = calculateSubtotal(size, toppings)
 discounted = applyDiscount(subtotal)
 final = applyCoupon(discounted, coupon)
 tax = final * 0.1
 return final + tax

SCOPE:

- basePrice = GLOBAL (constant)
- toppings = LOCAL to calculateSubtotal
- coupon = LOCAL to applyCoupon

INPUT:

- size (text from user)
- toppings (list from user)
- coupon (text from user, optional)

OUTPUT:

- "Order Summary"
- "Size: " + size
- "Toppings: " + toppings
- "Subtotal: \$XX.XX"
- "Discount: -\$XX.XX"
- "Tax: \$XX.XX"
- "Total: \$XX.XX"

ALGORITHM:

1. Get size from user
2. Get toppings from user
3. Get coupon from user (optional)
4. subtotal = calculateSubtotal(size, toppings)
5. discounted = applyDiscount(subtotal)

6. `final = applyCoupon(discounted, coupon)`
7. `tax = final * 0.1`
8. `finalTotal = final + tax`
9. Print order summary

ERROR HANDLING:

- Invalid size: default to `"small"`
- Empty toppings: `no` extra charge
- Invalid coupon: ignore it
- Negative price: impossible (data validation)

15:00 - 15:30 | FINAL WRAP-UP & CERTIFICATES

[SAY] "Congratulations! You've completed the 5-day programming course. Let's recap EVERYTHING."

[WRITE ON WHITEBOARD - Full Summary]

COMPLETE PROGRAMMING FOUNDATION
DAY 1: DATA & OPERATORS - Data Types: Number, Text, Boolean, List, Nothing - Variables: Named <code>values</code> (immutable <code>in</code> FP) - Operators: Actions <code>on data</code>
DAY 2: EXPRESSIONS, STATEMENTS & CONDITIONS - Expressions: Questions that produce <code>values</code> - Statements: Commands that perform actions - Conditions: <code>IF/THEN/ELSE</code> decisions
DAY 3: LOOPS & DATA STRUCTURES - Loops: <code>FOR EACH</code>, <code>WHILE</code>, <code>REPEAT N</code> - Lists: Ordered collections - Records: Named <code>fields</code>
DAY 4: FUNCTIONS, SCOPE & ERRORS - Functions: Named logic packages - Scope: <code>Variable</code> visibility - Errors: Syntax, Runtime, Logic
DAY 5: INPUT/OUTPUT & ALGORITHMS - Input: <code>Data from</code> outside world - Output: <code>Data to</code> outside world - Algorithms: Step-by-step problem solving
FINAL RULE:

```
Program = INPUT → PROCESS (expressions) → OUTPUT
Only INPUT/OUTPUT are statements.
Everything else is expressions.
```

[SAY] "Let's do a final Q&A."

Common Questions & Answers:

Q: "Why can't I change variables?"

A: "Immutability creates history and predictability. You can always trace back."

Q: "Why avoid statements?"

A: "Statements have side effects. They make testing harder."

Q: "When do I use loops vs functions?"

A: "Loops repeat actions. Functions package logic. Use both."

Q: "What's the most important thing I learned?"

A: "Everything is just data flowing through functions."

15:25 - 15:30 | Certificate Distribution

[SAY] "Congratulations to all of you! You now understand the FOUNDATION of programming."

[DISTRIBUTE] Give each participant a completion certificate.

Next Steps Recommendation:

1. Practice writing "algorithms" for daily tasks
2. Try learning a programming language (start with Python or JavaScript)
3. Remember: Everything you learned applies to ALL programming languages

[FINAL CLOSING STATEMENT] "Programming is just: **Data goes in → Functions transform it → Data goes out.** You've learned the foundation. Everything else is just details."

"Thank you for your attention. Any final questions before we wrap up?"

Day 5 Facilitator Cheat Sheet

Mistake	How to Fix
Forgetting I/O	"Programs need to talk to the world. Always include input and output."

Mistake	How to Fix
Overcomplicating algorithms	"Break problems into SMALL steps."
Mixing logic with I/O	"Keep I/O at edges. Logic in middle."
Not handling errors	"What if user enters gibberish? Always handle errors."
Forgetting FP rules	"Review: expressions for logic, statements for edges, immutable variables."

Appendix: Answer Keys

Day 1 Answer Key

Exercise 1.1: Sort the Boxes

Number Box	Text Box	Boolean Box	List Box	Nothing Box
5	"Hello"	true	[1, 2, 3]	null
3.14	"John"	false	["a", "b", "c"]	undefined
-10	"123"	true	[true, false]	
0				

Exercise 1.2: Variable Lockers

Value	Variable	Change to	New Variable
"John"	name = "John"	"Jane"	newName = "Jane"
true	isActive = true	false	newIsActive = false
[1,2,3]	numbers = [1,2,3]	[1,2,3,4]	newNumbers = [1,2,3,4]
100	price = 100	90	newPrice = 90
"London"	city = "London"	"Paris"	newCity = "Paris"

Exercise 1.3: Calculator Game

Expression	Answer
$5 + 3 \times 2$	11

Expression	Answer
"Hello" + " " + "World"	"Hello World"
10 > 5 AND true	true
[1,2] + [3,4]	[1,2,3,4]
15 % 4	3
"Apple".length > 3	true
10 == "10"	true
10 === "10"	false
NOT (true AND false)	true
(5 + 5) × (2 + 3)	50

Exercise 1.4: Spot the Type Error

Expression	Error	Why?
"Hello" * 3	Yes	Can't multiply text
5 + true	Yes	Can't add number and boolean
[1,2] + 3	Yes	Can't add list and number
10 / 0	Yes	Can't divide by zero
"5" + 3	No	Works (type coercion)

Day 2 Answer Key

Exercise 2.1: Expression or Statement?

Item	E or S	Why?
Add 5 and 3	E	Produces a value
Print "Hello"	S	Performs an action
What is the capital of France?	E	Produces a value
Write your name on the board	S	Performs an action
Calculate the area of a circle	E	Produces a value
Save this file	S	Performs an action

Item	E or S	Why?
Is 10 greater than 5?	E	Produces a value
Send an email	S	Performs an action
Find the length of "Hello"	E	Produces a value
Turn on the light	S	Performs an action

Exercise 2.2: Spot the Statement

Statements (circle): Lines 1, 2, 3, 5, 6 **Expressions (underline):** Line 4

Rewritten (FP Style):

- `subtotal = total of all item.prices`
- `discount = subtotal * 0.1`
- `finalTotal = subtotal - discount`
- `Print finalTotal`

Day 3 Answer Key

Exercise 3.3: Shopping Cart

Loop	Calculation	Total
Loop 1	<code>total = 0 + 1.00</code>	1.00
Loop 2	<code>total = 1.00 + 2.00</code>	3.00
Loop 3	<code>total = 3.00 + 3.00</code>	6.00
Loop 4	<code>total = 6.00 + 4.00</code>	10.00
Loop 5	<code>total = 10.00 + 5.00</code>	15.00

Final Total: \$15.00

Exercise 3.4: Student Record

Grades	Average
[85, 90, 78, 88]	85.25

Day 4 Answer Key

Exercise 4.1: Write Recipes

1. **FUNCTION** square(number):
 return number * number
2. **FUNCTION** isAdult(age):
 return age >= 18
3. **FUNCTION** max(a, b):
 IF a > b THEN return a
 ELSE return b
4. **FUNCTION** textLength(text):
 return text.length
5. **FUNCTION** sumAll(list):
 total = 0
 FOR each number in list:
 total = total + number
 return total

Exercise 4.4: Fix the Broken Recipes

Error 1: Syntax Fix: `FUNCTION double(number):`

Error 2: Runtime Fix: `return total / length(scores)`

Error 3: Logic Fix: `return price * 0.9`

Day 5 Answer Key

Exercise 5.1: Algorithm Steps

Problem 1: Find Youngest

1. Look at first person → call them "youngest"
2. Look at next person
3. If next person is younger than "youngest":
4. Update "youngest" = this person
5. Repeat from step 2 until no more people
6. Output "youngest"

Problem 2: Find Number 10

1. For each number in list:
2. If number == 10:
3. Return true

4. After checking all numbers:
5. Return `false`

Problem 3: Calculate Average

1. `total = 0`
2. For each number in `list`:
3. `total = total + number`
4. `count = length of list`
5. `average = total / count`
6. Output average

Exercise 5.3: Complete Program Design

Test Cases:

Test Case 1:

- tickets = 3, no discounts
- Base Price: \$150
- Final Price: \$150

Test Case 2:

- tickets = 5, student, member
- Base Price: \$250
- Student Discount: -\$50
- Group Discount: -\$25
- VIP Upgrade: Yes
- Final Price: \$175

Test Case 3:

- tickets = 6, senior, not member
- Base Price: \$300
- Senior Discount: -\$45
- Group Discount: -\$30
- VIP Upgrade: No
- Final Price: \$225

Appendix: Props & Setup

Day-by-Day Props Checklist

Day 1 Props:

- ☐ 5 cardboard boxes (labeled: Numbers, Text, Booleans, Lists, Nothing)
- ☐ Bags with mixed items (5 numbers, 5 text, 5 booleans, 5 lists, 3 nothing)
- ☐ Index cards (100)
- ☐ Markers
- ☐ Whiteboard
- ☐ Flip chart paper
- ☐ Timer

Day 2 Props:

- ☐ Green/Red cards (30 each)
- ☐ Play money (\$1, \$5, \$10, \$20)
- ☐ Index cards (100)
- ☐ Whiteboard markers
- ☐ Timer

Day 3 Props:

- ☐ Name cards (10 with names)
- ☐ Secret password card
- ☐ Product cards (prices)
- ☐ Shopping cart props (basket, items)
- ☐ Index cards (100)
- ☐ Whiteboard markers
- ☐ Timer

Day 4 Props:

- ☐ Boxes for scope game (3 boxes, labeled)
- ☐ Blindfolds (5)
- ☐ Index cards (100)
- ☐ String/yarn (20m)
- ☐ Whiteboard markers
- ☐ Timer

Day 5 Props:

- ☐ Flip chart paper (10 sheets)
- ☐ Markers (assorted)

- [] Index cards (100)
- [] Pizza shop scenario cards
- [] Certificates (printed)
- [] Whiteboard markers
- [] Timer

Classroom Setup Diagrams

U-Shape Configuration (Days 1-2):

- Chairs around tables facing whiteboard
- Facilitator at front near whiteboard
- Props table at side

Open Floor Space (Day 3):

- Tables pushed to edges
- Open space in center
- Facilitator in middle

Group Tables (Days 4-5):

- 4-5 tables, 3-4 participants each
- Facilitator moves between groups
- Project board for presentations

Emergency Backup Plan

If exercises run short:

- Prepare additional practice problems
- Use "bonus" questions at end of each exercise

If exercises run long:

- Cut discussion time
- Reduce presentation time
- Move non-essential content to homework

If technology fails:

- Use whiteboard instead of projector
- Print materials in advance
- Use physical props instead of digital

If participants are disengaged:

- Change activity type (theory → physical → group)
- Call on individuals
- Use more role-play

END OF FACILITATOR EDITION**Master Index**

Topic	Candidate Edition Page	Facilitator Edition Page
Course Overview	3	3
Day 1 - Data, Variables & Operators	5	25
Day 2 - Expressions, Statements & Conditions	15	45
Day 3 - Loops, Lists & Data Structures	25	65
Day 4 - Functions, Scope & Errors	35	85
Day 5 - Input/Output, Algorithms & Design	45	105
Final Assessment	55	-
Answer Keys	-	125
Props & Setup	-	135

END OF COMPLETE COURSE BOOKLET